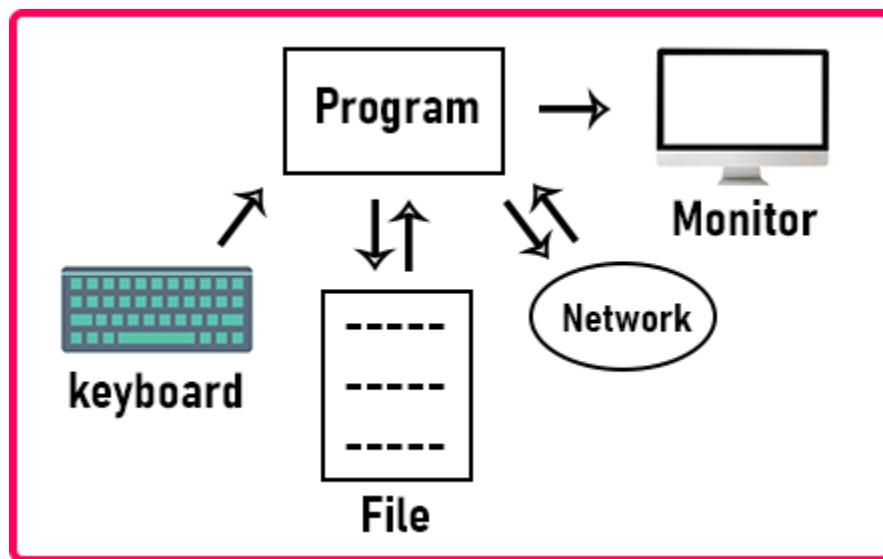


File Handling in C++

Before learning File Handling in C++, let us first learn about Streams in C++. **A stream is a flow of data or a flow of characters.**

Streams are a way to perform input and output operations in C++.

They are objects that allow you to read or write a sequence of characters, **typically from or to a file**. Streams are used to perform these operations in a flexible and efficient manner.



If we have a program, then the program may be getting the data from the keyboard or program may be sending the data to the monitor or the program may be accessing the data from an external file.

istream: Represents an input stream and is used to read data from an input source (e.g., a file or the console).

ostream: Represents an output stream and is used to write data to an output destination (e.g., a file or the console).

iostream: Represents an input/output stream and can be used to both read and write data.

Now similarly, for file access also, there are classes available that are ifstream for input stream and ofstream for the output stream.

For input stream from the keyboard, we have already a built-in object present in iostream header file that is cin and also an extraction operator that is ">>". So directly we can use cin with extraction operator. **Cin is an object of istream class.** And we have another object that is cout that we commonly used for displaying the text on the screen. **cout is an object of ostream.** cout is used with the insertion operator "<<".

```
#include <fstream>
int main(){
    ofstream outfile("my.txt");
}
```

So, this object outfile will associate with the file my.txt. Now whatever you will write into this outfile object it will get dropped into that file. **It's like you have connected a pipe from your program to that file on the disk. So, whatever you drop in the outfile it will get stored in the file.**

Next thing, when we were opening this file through outfile object, the file should be already existing. **If it is existing then outfile will open the file and if it is not existing then it will create a new file with the name my.txt.**

Next point, if the already file is existing and it is **having some content then what will happen to that content?** The outfile will truncate that content or remove the content. Suppose my.txt has some content, my.txt: 100 200 300

So, we have these numbers in the my.txt file. **These values will be removed and fresh content will be written.** If you want to append the new content after the old content then there is a mode available that is **ios::app**. 'app' stands for append.

So like this mode, there are two modes available that are **ios::app** and **ios::trunc**. 'trunc' stands for truncate.

After writing the content, you must close the file by writing the following statement. `outfile.close();`

This statement will close the stream and the file will be free from the program. It is important to close the file.

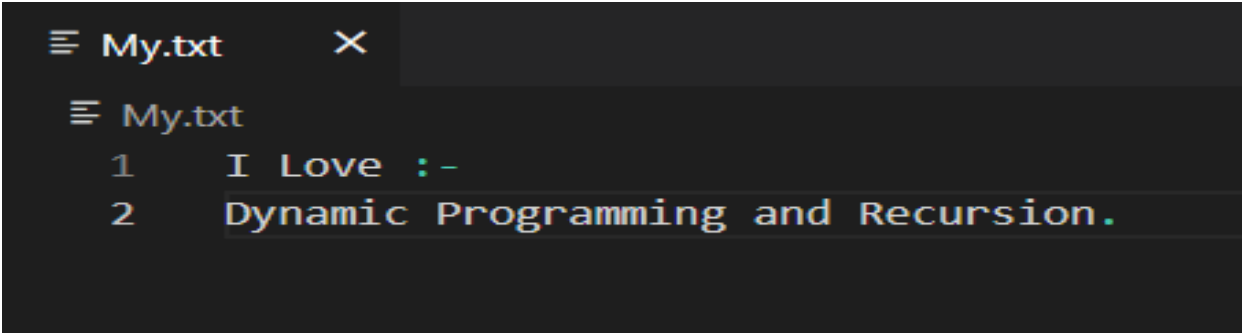
See sometimes, you have connected a pen drive or memory card to your laptop and you have kept one of the files open and you try to eject the card. Then you will get a message that "some program is using your memory card or pen drive". So, you cannot eject it. It means the operating system knows that some program is using some file so it will not allow you to eject. **It means that files are being occupied or that resource is in use.** So, when the program is not used then it should be released. If the program ends, then

automatically the resource will be released. So, it is a good practice to close the resource when you have finished.



```
1 #include<iostream>
2 #include<fstream>
3 using namespace std;
4 int main(){
5     ofstream outfile("My.txt", ios::trunc);
6     outfile<<"I Love :- "<<endl;
7     outfile<<"Dynamic Programming and Recursion."<<endl;
8     outfile.close();
9     return 0;
10 }
```

Output:-



```
My.txt
1 I Love :-
2 Dynamic Programming and Recursion.
```

Now, if you don't want to remove the old data instead you want to append the new data below the old data, then you need to use the `ios::app`

```
1 #include<iostream>
2 #include<fstream>
3 using namespace std;
4 int main(){
5     ofstream outfile("My.txt", ios::app);
6     outfile<<"And Competitive Programming As well"<<endl;
7     outfile.close();
8     return 0;
9 }
```

Output:-

```
≡ My.txt  X
≡ My.txt
1 I Love :-
2 Dynamic Programming and Recursion.
3 And Competitive Programming As well
4
```

How to Open a File in C++?

```
int main() {
    ifstream infile;
    infile.open("my.txt");
    if (!infile)
        cout << "File Cannot be opened";
}
```

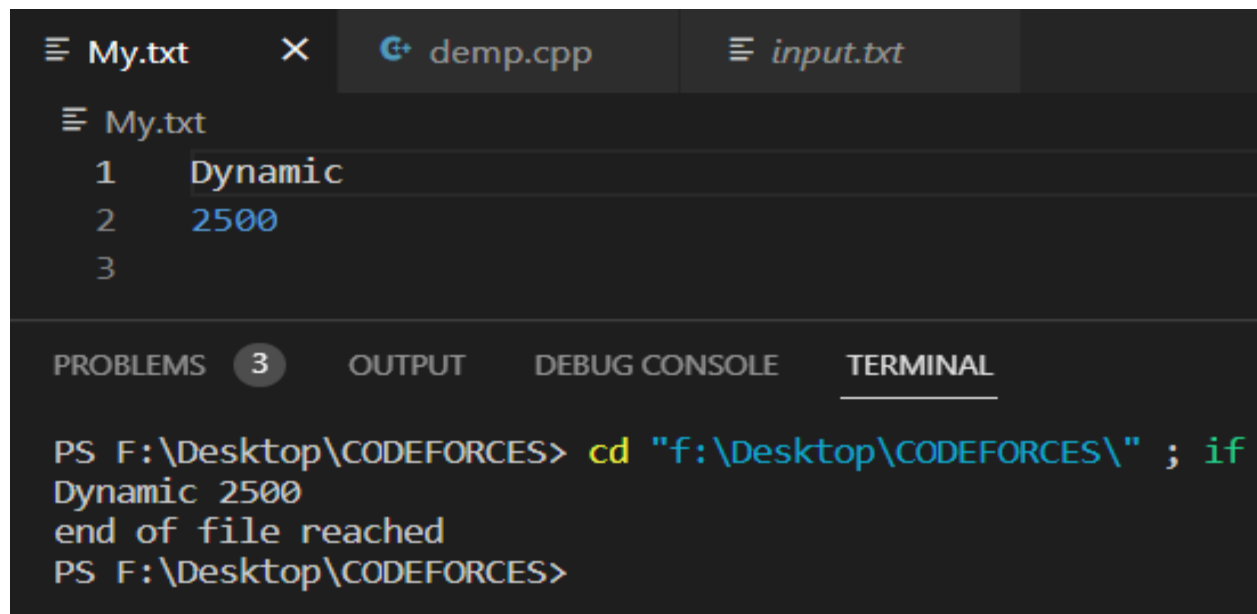
So, the `infile.open("my.txt")` statement will open the file. Now one important thing, when you are reading from a file then the file must be existing. It will not create a new file so the file must be existing at the location where the program is saved.

Next, we have checked for the file status. `if(!infile)` statement will check for whether the file is open or not.

There is another method for checking the file status.

`infile.is_open()`, this method will return true if the file is opened otherwise it will return false.

```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4  int main(){
5      ifstream infile;
6      infile.open("my.txt");
7      string str;
8      int x;
9      infile>>str;
10     infile>>x;
11     infile>>x;
12     cout<<str<<" "<<x<<endl;
13     if (infile.eof()){
14         cout<<"end of file reached";
15     }
16     infile.close();
17 }
```



```
My.txt x demp.cpp input.txt
My.txt
1 Dynamic
2 2500
3

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if
Dynamic 2500
end of file reached
PS F:\Desktop\CODEFORCES>
```

What is eof function in C++?

One important thing, once we have finished reading the content, we have reached the end of the file (eof). So sometimes we need to check whether we have reached the end or not. `infile.eof()`, this statement will return true if we are at the end of the file. So, we have written the print statement "end of file reached".

Example to Understand File Reading in C++:

```

1  #include<iostream>
2  #include<fstream>
3  using namespace std;
4  int main(){
5      ifstream ifs;
6      ifs.open ("my.txt");
7      if (ifs.is_open())
8          cout<<"File is Opened"<<endl;
9      string name;
10     int roll;
11     string branch;
12     ifs >> name >> roll >> branch;
13     ifs.close();
14     cout<<"Name: "<<name<<endl;
15     cout<<"Roll: "<<roll<<endl;
16     cout<<"Branch: "<<branch<<endl;
17     if(ifs.eof())
18         cout<<"End of File Reached";
19 }

```

My.txt x demp.cpp input.txt

My.txt

```

1  Aman_Babu
2  2019017
3  CSE
4

```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL

```

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ demp.cpp
File is Opened
Name: Aman_Babu
Roll: 2019017
Branch: CSE
End of File Reached
PS F:\Desktop\CODEFORCES>

```

Serialization in C++ :- Serialization is the process of storing and retrieving the state of an object. Suppose we want to store the student information in a file and again we want to retrieve the information of a student from a file. While working with Serialization the class must have a default constructor.

```
1  #include<iostream>
2  #include<fstream>
3  using namespace std;
4  class Student {
5  public:
6      string name;
7      int rollno;
8      string branch;
9      Student(){}
10     Student(string s, int roll_no, string t){
11         name = s;
12         rollno = roll_no;
13         branch = t;
14     }
15     friend ofstream & operator << (ofstream & ofs, Student s);
16     friend ifstream & operator >> (ifstream & ifs, Student & s);
17 };
18 int main(){
19     Student s1("Aman Babu", 2019017, "CSE");
20     ofstream ofs("Student.txt", ios::trunc);
21     ofs<<s1.name<<endl;
22     ofs<<s1.rollno<<endl;
23     ofs<<s1.branch<<endl;
24 }
```

≡ Student.txt X

≡ Student.txt

```
1  Aman Babu
2  2019017
3  CSE
4
```

Why we have stored all the values individually? If we can store the complete object with name, rollno and branch then we can say that we have stored the object. So, **if we can store the whole object at once then we can store an object as well as we can retrieve the object.** Yes. This is possible. We have to overload the operator of the ofs stream in the student class

```
1  #include<iostream>
2  #include<fstream>
3  using namespace std;
4  class Student{
5  public:
6      string name;
7      int rollno;
8      string branch;
9      Student(){}
10     Student(string n, int r, string b){
11         name = n;
12         rollno = r;
13         branch = b;
14     }
15     friend ofstream & operator<<(ofstream &ofs, Student s);
16     friend ifstream & operator>>(ifstream &ifis, Student &s);
17 };
18 ofstream & operator<<(ofstream & ofs, Student s){
19     ofs<<s.name<<endl;
20     ofs<<s.rollno<<endl;
21     ofs<<s.branch<<endl;
22     return ofs;
23 }
24 int main(){
25     Student s1("Aman Babu", 2019017, "CSE");
26     ofstream ofs("Student.txt", ios::trunc);
27     ofs<<s1;
28     ofs.close ();
29 }
```

≡ Student.txt ✕

≡ Student.txt

1 Aman Babu

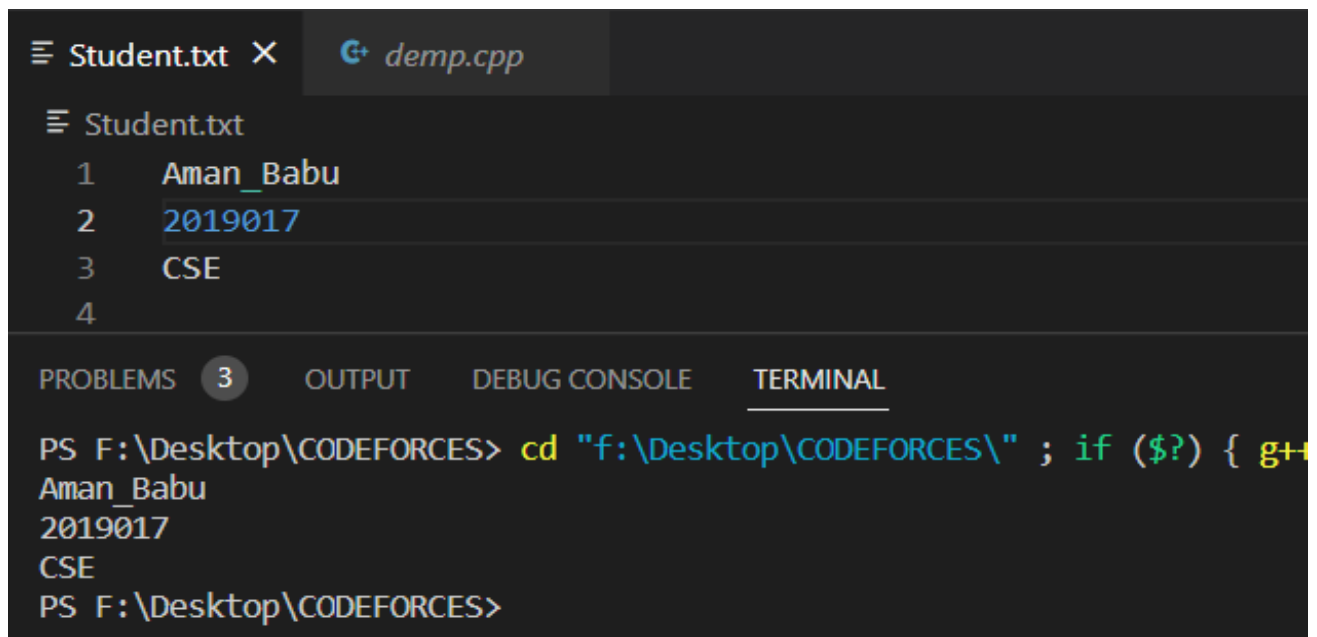
2 2019017

3 CSE

4

Retrieving the State of an Object in C++:-

```
1  #include<iostream>
2  #include<fstream>
3  using namespace std;
4  class Student{
5  public:
6      string name;
7      int rollno;
8      string branch;
9      Student(){}
10     Student(string n, int r, string b){
11         name = n;
12         rollno = r;
13         branch = b;
14     }
15     friend ofstream & operator<<(ofstream &ofs, Student s);
16     friend ifstream & operator>>(ifstream &if, Student &s);
17 };
18 ofstream & operator<<(ofstream & ofs, Student s){
19     ofs<<s.name<<endl;
20     ofs<<s.rollno<<endl;
21     ofs<<s.branch<<endl;
22     return ofs;
23 }
24 ifstream & operator >> (ifstream & ifs, Student & s){
25     ifs>>s.name>>s.rollno>>s.branch;
26     return ifs;
27 }
28 int main(){
29     Student s1("Aman_Babu", 2019017, "CSE");
30     ofstream ofs("Student.txt", ios::trunc);
31     ofs<<s1;
32     ofs.close ();
33     Student s2;
34     ifstream ifs("Student.txt");
35     ifs>>s2;
36     cout<<s2.name<<endl<<s2.rollno<<endl<<s2.branch<<endl;
37     ifs.close();
38 }
```



The screenshot shows a code editor with two tabs: 'Student.txt' and 'demp.cpp'. The 'Student.txt' tab is active, displaying a text file with four lines: 'Aman_Babu', '2019017', 'CSE', and an empty line. Below the editor, the 'TERMINAL' tab is active, showing the execution of a C++ program. The program reads the contents of 'Student.txt' and prints them to the console. The output matches the contents of the text file: 'Aman_Babu', '2019017', 'CSE', and a blank line.

```
Student.txt
1 Aman_Babu
2 2019017
3 CSE
4

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++
Aman_Babu
2019017
CSE
PS F:\Desktop\CODEFORCES>
```

Text and Binary Files in C++:

There are two types of files:

- Text Files
- Binary Files

Difference between Text and Binary Files in C++ with Examples:

Text files are human-readable whereas binary files are machine-readable.

Suppose we write a number that is 13. Then how is it written in the text file and binary file? Let's see the difference.

13 is an integer value. The binary form of 13 is 1101. So how many bytes do integers take? In most compilers, an integer takes 4 bytes but to make our explanation easy, we consider an integer takes 2 bytes. So, suppose 13 (integer) is taking 2 bytes then how many bits are there in 1101? **0000 0000 0000 1101**

Total 16 bits of a binary number. So, the same binary form is stored in the binary file with all 16 bits. That's why we called this a binary file.

Then what are text files? 13 will not be written in the text file. it will convert into ASCII. **The ASCII code of 1 is 49 and for 3 the ASCII code is 51.** Then what is the binary form of 49 and 51?

49 - 110001, 51 - 110011

These are the binary forms of 49 and 51. ASCII codes take 8 bits of binary so, **49 - 00110001, 51 - 00110011**

0011 0001 0011 0011 (This will be stored in the text file that is)

(ASCII code of 1) + (ASCII code of 3) = 49 + 51 =

(binary code of 49) + (binary code of 51) = 0011000100110011

Both text and binary files stored the binary number. Then how text files are human-readable?

Suppose we have a text file that has stored 13. When we open this file in notepad then what will notepad do? For every 8 bits, it will be converted into ASCII and then display that symbol.

0011 0001 0011 0011

So, for this binary number, the first 8 bits will be converted into ASCII which is 1 and the next 8 bits will be converted into ASCII which is 3. So, 1 and 3 will be displayed in a notepad.

And what about binary files? Suppose we have a binary file that contains the following code.

0000 0000 0000 1101

In a binary file, the first 8 bits will be taken. **In this case, all the bits are 0, and 0 is the ASCII code of some unknown garbage symbol. So, we will get some junk characters.** If you open this file in notepad then it might not show any meaningful symbols because the first 8 bits are not making any meaningful ASCII code. That's it. We cannot read and understand it because that is a pure binary form or we can say it is machine-understandable.

If you are reading from the text file then you can use the insertion and extraction operator for reading and writing the data in the form of text.

And if you want to read and write it in the form of binary then the first thing in C++ that you have to use is **iso::binary mode**. And also there are functions available for reading and writing that are **read()** and **write()**.

read() is available in the file input stream and **write()** is available in the file output stream.

Which file is faster? Binary file or text file?

The binary file is faster than the text file. Because text file needs conversion (symbols to ASCII and ASCII to binary). But in the binary file, there is no conversion required.

Which file takes more space?

Text files will take more space and binary will take less space.

Suppose we have the 4-digit number so a text file will take 4 bytes but a binary file will take 2 bytes.

Manipulators in C++

Manipulators are helping functions in C++ that are used to modify the input/output stream. What it means, **it will not modify the value of a variable, it will only modify the streams or format of streams using the insertion (<<) and extraction (>>) operators.**

- Manipulators are special functions that can be included in the I/O statement to alter the format parameters of a stream.
- Manipulators are operators that are used to format the data display.
- To access manipulators, the file `iomanip` should be included in the program.

Manipulators are used for enhancing streams or formatting streams. For writing the data, we can adopt some formats. For example, a common manipulator that we used is the `endl` that is used for the newline. Instead of `endl`, we can also say that `cout << "\n";`

This will also print a new line. So, `endl` is a manipulator which is used for formatting stream. So, it is useful for formatting output streams.

Integer Manipulators in C++:-

hex - It will display the data in hexadecimal.

oct - It will display data in the octal form.

dec - To display data in decimal form.

Float Manipulators in C++:-

fixed: It will show in the fixed floating-point number.

scientific: It will display the number in scientific form or exponent form.

There are other manipulators also available.

setw - It will set some amount of space for displaying the data. For example,

```
1  #include<iostream>
2  #include<iomanip>
3  using namespace std;
4  int main(){
5      cout<<"Hex 163: "<<hex<<163<<"\n";
6      cout<<"Oct 163: "<<oct<<163<<"\n";
7      cout<<"Dec 163: "<<dec<<163<<"\n";
8      cout<<"Fixed Manipulator: "<<fixed<<162.6454 <<endl;
9      cout<<"Scientific Manipulator: "<<scientific<<162.6454<<"\n";
10     cout<<setw(10)<<"World";
11     return 0;
12 }
```

Output:-